

FLIM Analysis Software Manual

Version 2.0 - Architecture, Methodology, Modules, Configuration, and Outputs

Research software documentation

Prepared from the supplied Python source files with the help of ChatGPT

Contents

1	Introduction
2	Software Architecture
3	Core Data Structures
4	Mathematical and Analytical Methodology
5	Module Reference
5.1	<code>__init__.py</code>
5.2	<code>read_ptu.py</code>
5.3	<code>extract_decay.py</code>
5.4	<code>extract_bin_decay.py</code>
5.5	<code>preprocess_decay.py</code>
5.6	<code>fit_decay.py</code>
5.7	<code>generate_maps.py</code>
5.8	<code>save_maps.py</code>
5.9	<code>export_tau_distribution.py</code>
5.10	<code>export_snr_vs_photons.py</code>
5.11	<code>export_selected_decays.py</code>
5.12	<code>run_analysis.py</code>
6	Configuration Reference
7	Output Files
8	Execution Guide
9	Quality Control and Diagnostics
10	Limitations and Assumptions
11	Recommended Future Development
12	Summary

1. Introduction

This document describes a modular Python workflow for processing fluorescence lifetime imaging microscopy (FLIM) measurements recorded in PicoQuant PTU format. The software converts raw time-correlated single-photon counting (TCSPC) image data into spatially resolved fluorescence-decay parameter maps, fit-quality diagnostics, photon-statistics outputs, selected decay curves, and reproducible analysis records.

The software is designed around an analysis-bin concept. A detector pixel is the smallest possible analysis bin, but adjacent pixels can be pooled into larger rectangular bins to improve photon statistics. Every analysis bin produces one summed fluorescence-decay histogram, which is then preprocessed, fitted, quality assessed, and stored at the corresponding location in the output maps.

1.1 Primary capabilities

- Import of PicoQuant PTU lifetime-imaging data.
- Handling of three-dimensional detector-position by TCSPC-time histogram data.
- Configurable rectangular regions of interest and spatial binning.
- Background estimation, subtraction, normalization, and temporal alignment of decay curves.
- Bounded nonlinear fitting using a stretched-exponential decay model.
- Configurable fitting-window selection, including noise-dependent stopping criteria.
- Photon-count, SNR, R-squared, RMSE, and fit-acceptance diagnostics.
- Generation of physical-parameter, signal-quality, and processing-diagnostic maps.
- Batch processing of one or more PTU measurements.
- Automated export to Origin-compatible text files, NumPy archives, and human-readable reports.

1.2 Workflow summary

```
PTU file selection
-> PTU decoding and metadata extraction
-> ROI and spatial-bin validation
-> Decay extraction and photon pooling
-> Background correction and normalization
-> Fit-window selection
-> Stretched-exponential fitting
-> Quality assessment and acceptance
-> Parameter-map generation
-> Data export and reporting
```

2. Software Architecture

The project follows a modular architecture. Low-level modules perform narrowly defined operations, while `generate_maps.py` coordinates per-bin numerical analysis and `run_analysis.py` controls complete file selection, configuration, batch execution, and export.

2.1 Dependency structure

```
run_analysis.py
|-- read_ptu.py
|-- generate_maps.py
|   |-- extract_bin_decay.py
|   |-- preprocess_decay.py
|   `-- fit_decay.py
|-- save_maps.py
|-- export_tau_distribution.py
|-- export_snr_vs_photons.py
`-- export_selected_decays.py
```

extract_decay.py is a separate pixel-level utility.
__init__.py defines the analysis package.

2.2 Architectural principles

- Separation of concerns: importing, extraction, preprocessing, fitting, map generation, saving, and optional exports are implemented independently.
- Structured data exchange: modules communicate through dictionaries and NumPy arrays rather than global state.
- Failure isolation: individual decay-processing or fitting errors are recorded without necessarily stopping analysis of the complete image.
- Reproducibility: analysis settings and measurement metadata are stored with the numerical outputs.
- Downstream compatibility: text exports are formatted for external analysis and visualization software, particularly OriginPro.

3. Core Data Structures

3.1 ptu_data

The read_ptu() function converts one PTU file into a dictionary used throughout the project.

```
ptu_data = {
    "filename": str,
    "metadata": dict,
    "lifetime_histogram_cube": numpy.ndarray,
    "time_axis": numpy.ndarray
}
```

The lifetime histogram cube has the conceptual shape image row x image column x TCSPC time bin. Each spatial detector pixel therefore contains a complete photon-count-versus-time histogram.

3.2 Preprocessed decay

```
preprocessed = {
    "time": one-dimensional array,
    "intensity": one-dimensional array,
    "stats": {
        "background_mean": ...,
        "background_std": ...,
        "peak": ...,
        ...
    }
}
```

3.3 Fit result

```
fit_result = {
    "ok": bool,
    "A": float,
    "tau": float,
    "beta": float,
    "C": float,
    "r2": float,
    "rmse": float,
    "cutoff": float,
    "fit_end_index": int,
    "fit_end_time": float,
    "fit_time": array,
    "fitted_curve": array
}
```

3.4 analysis_result

generate_maps() returns a higher-level dictionary containing all maps, selected decay records, spatial geometry, and analysis settings. This object is passed to all saving and optional-export modules.

```
analysis_result = {
    "maps": dict of two-dimensional arrays,
    "selected_decays": list of decay records,
    "roi": dict,
    "bin_size": dict,
    "geometry": dict,
    "settings": dict
}
```

4. Mathematical and Analytical Methodology

4.1 Spatial photon pooling

For each analysis bin, the software sums the TCSPC histograms of all detector pixels contained within that bin. If a bin contains N detector pixels, the resulting decay is the point-by-point sum of the N individual histograms. Larger bins generally increase photon count and fitting stability at the cost of spatial resolution.

4.2 Background estimation and correction

The background is estimated from a configurable fraction of the tail of the decay after optional removal of final TCSPC bins. The mean of the selected tail is subtracted from the full curve. The standard deviation of the tail is retained as a measure of baseline noise.

$$\text{corrected}(t) = \text{raw_counts}(t) - \text{background_mean}$$

4.3 Normalization and temporal alignment

The background-corrected signal is divided by its positive maximum. The curve is then trimmed to begin at the last point corresponding to the normalized maximum, and the time axis is shifted so that this point occurs at $t = 0$.

$$\text{normalized}(t) = \text{corrected}(t) / \text{peak}$$

4.4 Signal-to-noise ratio

Signal-to-noise ratio is calculated from the background-corrected peak and the standard deviation of the estimated baseline.

$$\text{SNR} = \text{peak} / \text{background_std}$$

4.5 Stretched-exponential model

Each preprocessed decay is fitted to the following model:

$$I(t) = A * \exp[-(t / \tau)^{\beta}] + C$$

- A: fitted amplitude or fixed value of 1.
- tau: characteristic lifetime.
- beta: stretching exponent describing departure from a simple single-exponential decay.
- C: fitted constant offset or fixed value of 0.

4.6 Fitting-window selection

The software supports three fit-window modes: noise, fixed, and none. In noise mode, the cutoff is derived from the estimated background noise and corrected peak. In fixed mode, a common normalized cutoff is used. In none mode, the entire valid curve is fitted. For thresholded modes, the signal must remain below the cutoff for a specified number of consecutive bins before fitting stops.

$$\text{noise cutoff} = \text{noise_k} * \text{background_std} / \text{peak}$$

4.7 Fit-quality metrics

- R-squared quantifies the fraction of variance represented by the fitted curve.
- RMSE measures the average magnitude of fitting residuals in normalized-intensity units.
- Photon-count and optional R-squared criteria determine which bins are accepted for final quantitative output.
- Fit-end index, fit-end time, and cutoff maps preserve information about how the fitting interval was selected.

5. Module Reference

5.1. __init__.py

Purpose

Identifies the analysis directory as an importable Python package.

Inputs

- No runtime data input.

Processing

1. Allows package-qualified imports from the analysis directory.

Outputs

- No numerical output.

Role in the workflow

Provides package organization. It performs no scientific processing.

5.2. read_ptu.py

Purpose

Reads a PicoQuant PTU measurement and converts it into the multidimensional arrays and metadata required by the remaining modules.

Inputs

- Path to one PTU file.

Processing

2. Validate that the input file exists.
3. Open the file with ptufile.PtuFile.
4. Decode the lifetime-imaging data.
5. Remove singleton array dimensions.
6. Extract the TCSPC time coordinate.
7. Collect acquisition, image, resolution, and photon-count metadata.
8. Assemble the ptu_data dictionary.

Outputs

- PTU filename.
- Metadata dictionary.
- Three-dimensional lifetime histogram cube.
- Physical TCSPC time axis.

Role in the workflow

Provides the raw numerical and metadata foundation for all subsequent analysis.

Implementation notes

- The decoded histogram cube is expected to have two spatial axes and one TCSPC axis.
- Total detected photons are calculated by summing the complete histogram cube.

5.3. extract_decay.py

Purpose

Extracts a fluorescence-decay histogram from one individual detector pixel.

Inputs

- ptu_data dictionary.

- Pixel column coordinate x.
- Pixel row coordinate y.

Processing

9. Retrieve the lifetime histogram cube.
10. Validate x and y against the detector dimensions.
11. Return the complete TCSPC histogram at the selected pixel.

Outputs

- One-dimensional fluorescence-decay array.

Role in the workflow

Acts as a simple pixel-level utility. The Version 2 map workflow instead uses the more general analysis-bin extractor.

5.4. extract_bin_decay.py

Purpose

Extracts and spatially pools the fluorescence decay for one rectangular analysis bin.

Inputs

- ptu_data dictionary.
- Rectangular ROI.
- Analysis-bin height and width.
- Zero-based bin row and column coordinates.

Processing

12. Validate ROI boundaries.
13. Validate positive bin dimensions.
14. Require exact divisibility of the ROI by the bin dimensions.
15. Calculate the output analysis grid.
16. Validate the requested bin coordinate.
17. Convert bin coordinates to original detector-pixel coordinates.
18. Slice the lifetime histogram cube.
19. Sum the detector-pixel histograms within the bin.

Outputs

- Summed one-dimensional decay.
- Detector-pixel start and stop coordinates.
- Number of detector pixels pooled.

Role in the workflow

Defines the fundamental spatial fitting unit and enables the trade-off between spatial resolution and photon statistics.

5.5. preprocess_decay.py

Purpose

Converts one raw photon-count decay into a normalized and temporally aligned curve suitable for fitting.

Inputs

- Raw decay curve.
- Optional physical time axis.
- Background-tail fraction.
- Number of final TCSPC bins to remove.
- Tolerance for locating the normalized maximum.

Processing

20. Convert inputs to one-dimensional floating-point arrays.
21. Validate array lengths and configuration values.
22. Optionally remove final raw time bins.
23. Estimate background mean and standard deviation from the decay tail.
24. Subtract the background mean.
25. Verify that a positive signal remains.
26. Normalize by the corrected peak.
27. Locate the final point at the normalized maximum.
28. Trim the curve to start at the maximum.
29. Shift the retained time axis to $t = 0$.
30. Assemble preprocessing statistics.

Outputs

- Preprocessed time axis.
- Normalized intensity.
- Statistics used for SNR and fit-window selection.

Role in the workflow

Applies a consistent, repeatable signal-conditioning procedure to every analysis bin.

Implementation notes

- End-bin trimming occurs before background estimation to prevent acquisition artifacts from biasing the baseline.

5.6. fit_decay.py

Purpose

Fits one preprocessed fluorescence decay using a bounded stretched-exponential model and calculates fit-quality diagnostics.

Inputs

- Preprocessed time and intensity arrays.

- Optional preprocessing statistics.
- Threshold mode.
- Amplitude fitting option.
- Background fitting option.

Processing

31. Validate dimensionality and finite data.
32. Determine the fitting endpoint using noise, fixed, or full-curve mode.
33. Require a minimum number of fitting points.
34. Select the appropriate parameterization depending on fixed or fitted amplitude and background.
35. Perform bounded nonlinear optimization with `scipy.optimize.curve_fit`.
36. Calculate the fitted curve.
37. Calculate R-squared and RMSE.
38. Return structured failure reasons when fitting is not possible.

Outputs

- Fitted A, tau, beta, and C parameters.
- R-squared and RMSE.
- Fit cutoff and endpoint information.
- Fitted time axis and model curve.
- Success status or structured failure reason.

Role in the workflow

Provides the nonlinear numerical engine used by `generate_maps.py`.

Implementation notes

- Parameter bounds and initial estimates are currently defined as module-level configuration constants.
- The configured lifetime bounds are tailored to the intended experimental system and should be reviewed for other samples.

5.7. generate_maps.py

Purpose

Coordinates extraction, preprocessing, fitting, acceptance, and spatial storage for every analysis bin in one PTU measurement.

Inputs

- `ptu_data`.
- ROI and bin size.
- Optional selected-bin list.
- Photon and R-squared threshold settings.
- Preprocessing and fitting configuration.

Processing

39. Validate ROI, bin geometry, and selected-bin coordinates.

40. Initialize all parameter and diagnostic maps with NaN or zero masks.
41. Loop through every analysis bin.
42. Extract and sum the bin decay.
43. Calculate total and peak photon counts.
44. Apply the optional pre-fit photon threshold.
45. Preprocess the decay.
46. Calculate SNR.
47. Fit the decay.
48. Catch per-bin processing and fitting exceptions.
49. Store physical parameters and diagnostic values in spatial maps.
50. Evaluate the R-squared acceptance criterion.
51. Set the final success mask.
52. Retain detailed raw and fitted data for selected bins.
53. Return maps, records, geometry, and settings.

Outputs

- Lifetime, beta, amplitude, offset, R-squared, RMSE, photon-count, SNR, cutoff, endpoint, and status maps.
- Selected decay records.
- Spatial geometry and analysis settings.

Role in the workflow

Serves as the central computational coordinator and converts individual decay fits into spatially resolved FLIM outputs.

Implementation notes

- NaN initialization prevents failed bins from appearing as genuine zeros.
- The R-squared diagnostic map is distinct from the final success mask.

5.8. save_maps.py

Purpose

Saves FLIM maps, a complete NumPy archive, and a human-readable analysis report.

Inputs

- analysis_result.
- Complete run settings.
- Measurement metadata.
- Output directory.
- Measurement name.

Processing

54. Create the measurement output folder if needed.
55. Write every map as a tab-delimited matrix.
56. Create a compressed NPZ archive containing maps and settings.

- 57. Count successful and failed bins.
- 58. Write measurement, spatial, preprocessing, fitting, and output information to an analysis report.

Outputs

- One text matrix per map.
- Compressed NPZ archive.
- Human-readable analysis report.
- Dictionary of saved paths.

Role in the workflow

Provides traceable, reusable, and downstream-compatible storage of the results.

5.9. export_tau_distribution.py

Purpose

Exports accepted fitted lifetime values for distributional and statistical analysis.

Inputs

- analysis_result.
- Output directory.
- Measurement name.

Processing

- 59. Retrieve the lifetime map and final success map.
- 60. Select values only from accepted bins.
- 61. Remove non-finite values.
- 62. Sort the lifetime values.
- 63. Write one lifetime value per line in nanoseconds.

Outputs

- Text file containing the accepted lifetime distribution.

Role in the workflow

Supports histogram generation, distribution fitting, sample comparison, and analysis of spatial lifetime heterogeneity.

5.10. export_snr_vs_photons.py

Purpose

Exports the relationship between total photon count and signal-to-noise ratio across valid analysis bins.

Inputs

- analysis_result.
- Output directory.

- Measurement name.

Processing

64. Flatten the total-photon and SNR maps.
65. Remove non-finite pairs.
66. Sort rows by increasing photon count.
67. Write a two-column tab-delimited table.

Outputs

- Text file with Total_photons and SNR columns.

Role in the workflow

Enables evaluation of photon-statistics requirements, spatial-binning effects, and possible minimum-photon thresholds.

5.11. export_selected_decays.py

Purpose

Exports raw, processed, and fitted decay data for user-selected spatial bins.

Inputs

- analysis_result.
- Output directory.
- Measurement name.

Processing

68. Iterate through selected decay records.
69. Convert raw and fitted time axes to nanoseconds.
70. Retrieve raw counts, processed intensity, and fitted curve.
71. Pad arrays with NaN to obtain a common row count.
72. Write a five-column table.
73. Add spatial boundaries, pooled-pixel count, fit parameters, and quality metrics to the header.

Outputs

- One detailed text file per selected analysis bin.

Role in the workflow

Supports visual verification, representative-curve plotting, and diagnosis of problematic fits.

5.12. run_analysis.py

Purpose

Provides the top-level, user-executed controller for the complete Version 2 workflow.

Inputs

- Centralized analysis settings in the script.
- One or more PTU files selected through a Tkinter dialog.
- One output root folder selected through a Tkinter dialog.

Processing

74. Define software, spatial, threshold, preprocessing, fitting, and export settings.
75. Open file-selection and folder-selection dialogs.
76. Loop sequentially through all selected PTU files.
77. Read each PTU file.
78. Create a dedicated output folder for each measurement.
79. Record analysis time and all active settings.
80. Call `generate_maps()` with the selected configuration.
81. Save maps and reports when enabled.
82. Run optional lifetime, SNR, and selected-decay exports.
83. Validate that optional exports have an available `analysis_result`.
84. Print progress and completion summaries.

Outputs

- A structured output folder for every selected measurement.
- Complete map, archive, report, and optional export files.

Role in the workflow

Integrates all modules into a repeatable batch-processing workflow and is the main entry point used by the researcher.

Implementation notes

- Batch execution is sequential rather than parallel.
- The interface is partially graphical for file selection, while analytical settings are edited in the Python source.

6. Configuration Reference

The following settings are defined in `run_analysis.py` and control the complete workflow.

Setting	Function
<code>SOFTWARE_VERSION</code>	Software version recorded in the analysis report.
<code>VERBOSE</code>	Controls progress messages during map generation.
<code>ROI</code>	Rectangular detector-pixel region to analyze; stop coordinates are exclusive.
<code>BIN_SIZE</code>	Height and width of each analysis bin in detector pixels.
<code>SELECTED_BINS</code>	Zero-based bin coordinates retained for detailed decay export.
<code>APPLY_PHOTON_THRESHOLD</code>	Enables or disables pre-fit photon-count rejection.
<code>MIN_TOTAL_PHOTONS</code>	Minimum summed photon count required when photon thresholding is enabled.
<code>APPLY_R2_THRESHOLD</code>	Determines whether R-squared is only diagnostic or a final acceptance criterion.
<code>MIN_R2</code>	Minimum R-squared value used by the diagnostic and optional acceptance logic.

TRIM_END_BINS	Number of terminal TCSPC bins removed before background estimation.
BACKGROUND_FRACTION	Fraction of the remaining decay tail used to estimate the baseline.
RUN_MAP_ANALYSIS	Enables or disables the main analysis.
SAVE_MAPS	Controls map, archive, and report saving.
THRESHOLD_MODE	Selects noise, fixed, or none for fit-window determination.
FIT_AMPLITUDE	Fits A when true; fixes A = 1 when false.
FIT_BACKGROUND	Fits C when true; fixes C = 0 when false.
RUN_TAU_DISTRIBUTION_EXPORT	Controls accepted-lifetime export.
RUN_SNR_VS_PHOTONS_EXPORT	Controls photon-count versus SNR export.
RUN_SELECTED_DECAY_EXPORT	Controls detailed selected-bin decay export.

7. Output Files

7.1 Parameter and diagnostic maps

Map	Meaning
tau_ns	Characteristic lifetime in nanoseconds.
beta	Stretched-exponential exponent.
A	Fitted amplitude.
C	Fitted constant offset.
r2	Coefficient of determination.
rmse	Root-mean-square fitting error.
total_photons	Total raw photons in each analysis bin.
peak_counts	Maximum raw histogram count in each bin.
snr	Signal-to-noise ratio.
fit_end_index	Index at which the fit interval ends.
fit_end_time_ns	Physical fit endpoint in nanoseconds.
cutoff	Normalized threshold used for fit-window selection.
success	Final accepted-bin mask.
r2_accepted	Diagnostic mask indicating whether R-squared meets MIN_R2.

7.2 Archive and report

- <measurement>_maps.npz: compressed Python archive containing all maps, ROI, bin size, geometry, and run settings.
- <measurement>_analysis_report.txt: human-readable summary of measurement metadata, configuration, and fit statistics.

7.3 Optional exports

- <measurement>_tau_distribution.txt: sorted accepted lifetime values.
- <measurement>_snr_vs_total_photons.txt: two-column photon-count and SNR data.
- <measurement>_bin_yY_xX_decay_fit.txt: raw and fitted decay data for one selected bin.

8. Execution Guide

85. Install the required Python packages, including NumPy, SciPy, ptufile, and a Python installation with Tkinter support.
86. Place the processing modules inside an analysis package directory alongside __init__.py.
87. Review the model parameter bounds and initial estimates in fit_decay.py for the intended sample system.

88. Edit the analysis settings in `run_analysis.py`, including ROI, bin size, thresholds, preprocessing, and export options.
89. Run `run_analysis.py` from a Python environment in which the analysis package is importable.
90. Select one or more PTU input files in the file-selection dialog.
91. Select the root output directory.
92. Monitor the console for progress and errors.
93. Review the generated analysis report, success map, R-squared map, and representative selected-bin decay files before interpreting the lifetime maps.

8.1 Recommended first validation run

- Use a small ROI and moderate spatial bin size to reduce processing time.
- Export several representative bins, including bright, dim, and background regions.
- Inspect raw decay, processed intensity, and fitted curve together.
- Compare full-curve and noise-threshold fitting modes where appropriate.
- Check whether the configured lifetime and beta bounds are reached frequently.
- Use photon-count, SNR, R-squared, and RMSE maps to identify systematic failure regions.

9. Quality Control and Diagnostics

The workflow includes several complementary layers of quality control. These should be evaluated together rather than relying on one map or threshold.

- Total-photon map: identifies low-count regions and supports pre-fit filtering.
- SNR map: shows the relationship between corrected peak signal and baseline noise.
- R-squared map: measures agreement between the observed and fitted curve.
- RMSE map: measures the scale of normalized residuals.
- R-squared acceptance map: indicates whether each fit meets the selected criterion, even when the criterion is not enforced.
- Success map: indicates which bins are included in final quantitative outputs.
- Fit-end maps: reveal spatial variation in the amount of each decay used during fitting.
- Selected-decay export: allows direct visual inspection of raw data, preprocessing, fitting interval, and model agreement.
- Analysis report: records the settings required to reproduce and interpret the run.

10. Limitations and Assumptions

- The workflow assumes that decoded PTU data can be represented as a two-dimensional image with one TCSPC histogram per pixel.
- The ROI dimensions must be exactly divisible by the selected analysis-bin dimensions.
- The stretched-exponential model is assumed to be appropriate for the measured decays. Alternative models are not currently selectable from `run_analysis.py`.
- Parameter bounds and initial estimates are defined in `fit_decay.py` and are currently tailored to a particular experimental context.
- Background is estimated from the tail of each decay; this assumes that the selected tail adequately represents the baseline.

- Normalization removes absolute amplitude information from the fitted intensity curve, although raw photon-count maps are retained separately.
- R-squared and RMSE do not alone prove that the chosen decay model is physically correct.
- Batch processing is sequential and may be time-consuming for large images or small analysis bins.
- The user interface covers file and folder selection, but analytical settings are changed directly in the source code.
- The software records analysis settings but does not currently provide formal automated unit testing, uncertainty estimation, or validation against reference datasets within the supplied files.

11. Recommended Future Development

- Move model bounds, initial estimates, and fitting-window constants into a central configuration object or external configuration file.
- Add command-line arguments or a graphical settings interface to avoid direct editing of `run_analysis.py`.
- Add automated unit tests for geometry validation, preprocessing, fit-window logic, model fitting, and exports.
- Add synthetic-decay validation tests with known lifetimes, beta values, backgrounds, and photon noise.
- Add parameter-uncertainty estimates or confidence intervals from the covariance matrix or resampling methods.
- Add alternative decay models and formal model-comparison criteria.
- Add parallel processing for independent analysis bins or independent PTU files.
- Add logging to a file, including warnings and per-bin failure counts grouped by reason.
- Add plotting utilities for rapid inspection of maps, lifetime distributions, residuals, and selected decay fits.
- Package the software with dependency specifications, version control tags, and an installation guide.
- Add automated generation of a machine-readable manifest describing every output file.

12. Summary

The supplied modules constitute a genuine end-to-end scientific data-analysis workflow. The software imports proprietary FLIM measurements, transforms multidimensional photon-histogram data into configurable spatial analysis units, preprocesses and fits fluorescence decays, calculates signal and fit-quality diagnostics, generates quantitative spatial maps, and exports reproducible results for further analysis and visualization.

The strongest feature of the codebase is the integration of the full analytical sequence rather than any single algorithm. Data import, preprocessing, nonlinear modelling, quality control, spatial mapping, batch execution, and reporting are connected through a modular and traceable architecture.